

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ	1
ВВЕДЕНИЕ	2
1 ПОСТАНОВКА ЗАДАЧИ	3
2 ВЫПОЛНЕНИЕ ЗАДАНИЯ	4
2.1. Анализ предметной области	4
2.2. Проектирование БД Веб-сайта	5
2.3. Модуль авторизации	7
2.4. Модуль пользователя	9
2.5. Модуль лотов и ставок	10
2.6. Модуль администратора	11
2.7. Контейнеризация	12
ЗАКЛЮЧЕНИЕ	13
СПИСОК ИСТОЧНИКОВ	14
ПРИЛОЖЕНИЕ А. Диаграммы базы данных	15
ПРИЛОЖЕНИЕ Б. Исходный код приложения	17
ПРИЛОЖЕНИЕ В. Исходный код приложения	18

ВВЕДЕНИЕ

Актуальность темы обусловлена высоким спросом онлайн-аукционов как удобного способа купли-продажи товаров, обеспечивающих прозрачность торгов, надёжное хранение данных и гибкое управление. Современные веб-технологии позволяют создать платформу, доступную для широкой аудитории, которая будет открывать новые возможности для участников аукционов.

Целью работы является разработка онлайн-платформы аукциона, предоставляющей регистрацию и авторизацию пользователей, создание и управление лотами, участие в торгах с предложением цены, просмотр истории ставок и управление покупками. Система должна корректно обрабатывать следующие сценарии: запрет ставок на свой лот, отмену последней неперебитой ставки, выкуп или отказ при победе. Для администраторов предусматривается управление категориями. Реализация выполняется на платформе .NET, ASP.NET Core и Entity Framework Core с СУБД PostgreSQL, с развёртыванием в Docker.

Для реализации необходимо решить следующие задачи: провести анализ предметной области; спроектировать и нормализовать базу данных; разработать серверную логику (регистрация, управление лотами, ставки, завершение торгов, администрирование категорий); создать пользовательские интерфейсы; реализовать фильтрацию, поиск и сортировку; разработать личный кабинет; обеспечить ролевой доступ; выполнить тестирование и развернуть в Docker.

1 ПОСТАНОВКА ЗАДАЧИ

В рамках производственной практики требуется разработать онлайн-платформу для проведения аукционов. Система позволяет зарегистрированным пользователям создавать лоты с указанием начальной цены, описания, категории и срока действия, а также участвовать в торгах, предлагая свою цену. Победителем признаётся участник, предложивший наибольшую цену на момент закрытия лота, после чего он может выкупить лот или отказаться от покупки.

Функциональные требования включают регистрацию и аутентификацию с разграничением ролей (пользователь и администратор), просмотр каталога с фильтрацией по категориям, поиском, сортировкой и фильтрацией по времени окончания. На странице лота отображаются полная информация и история ставок, пользователь может предложить цену с проверками (выше текущей, не на свой лот, не дублировать). Предусмотрена отмена последней неперебитой ставки. После завершения торгов победитель выбирает выкуп или отказ. В личном кабинете доступны созданные лоты (просмотр, редактирование, удаление), история ставок и покупок. Администратор управляет категориями (создание, редактирование, удаление).

Технические требования: работа в современных браузерах, сервер на ASP.NET Core с Entity Framework Core, СУБД PostgreSQL, контейнеризация Docker, клиентская часть на HTML, CSS, JavaScript с HTMX.

2 ВЫПОЛНЕНИЕ ЗАДАНИЯ

2.1. Анализ предметной области

Аукцион представляет собой способ купли-продажи товаров или услуг, при котором покупатели соревнуются за право приобретения лота, предлагая последовательно повышающиеся цены. Продавец выставляет товар с начальной стоимостью, а участники делают ставки, увеличивая цену до тех пор, пока не будет определён победитель – тот, кто предложил максимальную сумму на момент закрытия торгов. Классическая модель аукциона предполагает открытые торги, где все участники видят текущие предложения и могут реагировать на них, что создаёт конкурентную среду и позволяет продавцу получить наиболее выгодную цену.

Наиболее распространённым является аукцион, в котором участники последовательно повышают цену, а лот достаётся тому, кто сделал последнюю и самую высокую ставку. Именно эта модель легла в основу разрабатываемой онлайн-платформы. Важной особенностью аукционов является временное ограничение – торги длятся в течение заданного периода, по истечении которого дальнейшие ставки не принимаются, и победитель определяется автоматически.

Система должна гарантировать достоверность информации о текущей цене и оставшемся времени, чтобы все участники находились в равных условиях. Необходимо соблюдать правила, исключающие возможность ставок на собственные лоты, а также предотвращать дублирование предложений от одного пользователя. После завершения торгов должна быть реализована чёткая процедура выкупа или отказа, чтобы продавец и покупатель могли завершить сделку. Для удобства пользователей требуется эффективная система поиска и фильтрации лотов по категориям, цене и времени окончания, а также личный кабинет, позволяющий отслеживать активность и историю участия.

Таким образом, разрабатываемая платформа должна реализовать все перечисленные функции в рамках веб-приложения, обеспечивая удобный интерфейс как для продавцов, так и для покупателей.

2.2. Проектирование БД Веб-сайта

В качестве системы управления базами данных выбрана PostgreSQL. Данное решение обусловлено рядом преимуществ: PostgreSQL является объектно-реляционной СУБД с открытым исходным кодом, поддерживает расширенные типы данных, обеспечивает высокую надёжность и соответствие стандартам SQL. Она хорошо интегрируется с платформой .NET и Entity Framework Core, предоставляя богатые возможности для миграций и работы с большими объёмами данных. Кроме того, PostgreSQL поддерживает сложные запросы, транзакционность и механизмы конкурентного доступа, что критически важно для корректной обработки ставок в условиях одновременных запросов от множества пользователей.

Модель данных включает следующие основные сущности. Пользователь (User) является центральной сущностью. Каждый пользователь может создавать множество лотов, делать множество ставок и совершать множество покупок. Лот (Lot) представляет собой выставляемый на торги товар, содержит информацию о названии, количестве, начальной и текущей цене, времени окончания торгов, описании, городе, способах оплаты и доставки, а также статус и признак закрытия. Каждый лот обязательно принадлежит одному пользователю-продавцу и относится к одной категории (Tag). С каждым лотом может быть связано несколько изображений (LotImage). Ставка (Bid) фиксирует предложение цены конкретным пользователем за конкретный лот. Покупка (Purchase) сохраняет информацию о завершённой сделке, когда победитель торгов выкупает лот по зафиксированной цене. Категория (Tag) служит для классификации лотов и имеет уникальное название.

Была построена ER-диаграмма «сущность-связь» в нотации П. Чена, которая позволяет представить базу данных на концептуальном уровне и определить атрибуты и логические связи сущностей. Диаграмма изображена рисунке А.1 в Приложении А.

Разработанная база данных состоит из шести таблиц, объединённых внешними ключами и обеспечивающих хранение всей необходимой информации для функционирования интернет-магазина. Ниже перечислены основные сущности и их назначение.

Таблица «users» содержит поля: login (строковый, первичный ключ), email (уникальный, допускает NULL), password_hash (обязательное, хранит хешированный пароль), is_admin (логический, по умолчанию false).

Таблица «tags» включает поля id (целочисленный, первичный ключ, автоинкремент) и name (обязательное, уникальное).

Таблица «lots» является наиболее объемной: id (первичный ключ), user_login (внешний ключ к users.login, продавец), tag_id (внешний ключ к tags.id), title, count (количество товара), initial_price и current_price (начальная и текущая цена, хранятся как decimal), end_time (время окончания торгов в формате DateTimeOffset), description, city, payment_method и delivery_payment (строковые, определяются перечислениями), leader_login (внешний ключ к users.login, ссылка на текущего лидера, может быть NULL), closed (логический, признак завершения торгов), status (строковое, отражает состояние лота). На таблицу lots также созданы индексы для ускорения запросов по внешним ключам и полям leader_login, tag_id, user_login.

Таблица «lot_images» служит для хранения изображений лотов и включает id (первичный ключ), lot_id (внешний ключ к lots.id с каскадным удалением) и image_path (путь к файлу изображения). Связь с лотом организована один-ко-многим, при этом допускается наличие нескольких изображений для одного лота, а первое изображение по порядку используется как миниатюра.

Таблица «bids» фиксирует ставки и содержит id (первичный ключ), user_login (внешний ключ к users.login), lot_id (внешний ключ к lots.id) и price (десятичное значение предложенной цены). Обе внешние связи настроены с каскадным удалением, что гарантирует удаление ставок при удалении пользователя или лота. Для ускорения выборок созданы индексы по полям lot_id и user_login.

Таблица «purchases» хранит информацию о покупках: id (первичный ключ), user_login (внешний ключ к users.login), lot_id (внешний ключ к lots.id), locked_price (цена, по которой была совершена покупка). Аналогично ставкам, связи настроены с каскадным удалением и индексируются.

На рисунке A.2 в Приложении A показана полная атрибутивная модель базы данных в нотации IDEF1X.

Все таблицы были составлены на языке C# и сгенерированы с помощью миграций и обновлений Entity Framework Core. С дополнительными атрибутами на полях и классах, уточняющие их назначение, позволили сгенерировать гибкую и удобную в использовании схему.

Все внешние связи между таблицами обеспечивают целостность данных на уровне СУБД, а использование индексов по внешним ключам и уникальных ограничений (на email пользователя и имя категории) гарантирует производительность и отсутствие дублирования критически важных значений. В результате была получена логически упорядоченная структура базы данных, которая соответствует требованиям сайта магазина музыкальных инструментов.

2.3. Модуль авторизации

Модуль авторизации обеспечивает защищённый доступ к приложению, реализуя регистрацию, вход и управление сессией. Аутентификация построена на основе JWT-токенов, которые генерируются после успешной проверки учётных данных и сохраняются в cookies клиента. Токен содержит информацию о пользователе (логин, роль) и подписывается секретным ключом, что гарантирует его целостность и невозможность подделки. Для повышения безопасности доступ к защищённым маршрутам требует наличия валидного токена, который проверяется на каждом запросе через middleware. При истечении срока действия основного токена система автоматически пытается обновить его с использованием refresh-токена, хранящегося в отдельной cookie и в базе данных. Фоновая служба периодически очищает просроченные refresh-токены, предотвращая их накопление. Обновление происходит незаметно для пользователя: клиент получает новую пару токенов при запросе к специальному эндпоинту.

Регистрация и вход реализованы через стандартные HTML-формы. При регистрации нового пользователя система принимает два параметра: адрес электронной почты и пароль. Перед сохранением в базу данных пароль подвергается криптографической обработке. На первом этапе генерируется уникальная криптографическая соль (salt). Эта соль комбинируется с введенным пользователем паролем, после чего полученная строка многократно пропускается через алгоритм хеширования PBKDF2 с использованием SHA256. Полученный хеш вместе с солью сохраняется в таблице Users вместо исходного пароля. Когда пользователь отправляет запрос на аутентификацию, система извлекает из базы данных соль, соответствующую указанному email. Используя эту соль и предоставленный пароль, сервер выполняет идентичную процедуру хеширования с теми же параметрами. Сравнение полученного хеша с хранимым в базе значением происходит через константную по времени функцию, что исключает возможность проведения timing-атак. Только при полном совпадении хешей доступ считается подтвержденным.

Валидация форм выполняется как на клиентской стороне, так и на серверной: проверяется уникальность логина и почты, совпадение пароля и его подтверждения, а также заполненность обязательных полей. Для улучшения пользовательского опыта реализована динамическая проверка существования логина или почты через HTMX-запросы, которые выводят сообщения об ошибках без перезагрузки страницы. При успешной аутентификации пользователь перенаправляется на ранее сохранённый URL или на главную страницу.

Если токен отсутствует или истёк, а пользователь не успел обновить его, система автоматически перенаправляет с защищённых маршрутов на страницу входа, сохраняя при этом адрес запрошенной страницы для последующего возврата. Выход из системы осуществляется путём удаления токенов из cookies и очистки refresh-токена в базе.

Таким образом реализован функционал регистрации и авторизации пользователя.

2.4. Модуль пользователя

Модуль пользователя предоставляет интерфейс для просмотра и управления личной информацией, а также доступ к данным о созданных лотах, ставках и покупках. Публичная часть модуля доступна всем посетителям и показывает логин пользователя и список его лотов (без возможности редактирования). Для авторизованного пользователя открывается полный личный кабинет, где он может увидеть свои лоты с возможностью перехода к редактированию или удалению, а также отдельные вкладки со списком своих ставок и покупок. Каждая вкладка загружается динамически с помощью HTMX, что обеспечивает быстрый отклик без полной перезагрузки страницы.

Через этот модуль пользователь может управлять созданными лотами: переходить на страницу редактирования или удалять лот. Удаление сопровождается физическим удалением файлов изображений с диска и записей из базы. Для удобства в личном кабинете отображается миниатюра каждого лота, его название и текущая цена. В разделе ставок пользователь видит все лоты, на которые он делал ставки, с указанием своей ставки и статуса: если он является лидером, отображается соответствующая метка; если его перебили – показывается текущая цена и уведомление. В разделе покупок перечислены лоты, которые пользователь выкупил (статус Purchased), с возможностью перехода к их странице. Кроме того, модуль предоставляет возможность смены пароля: пользователь вводит старый и новый пароль, после чего выполняется проверка старого пароля, и при успехе хеш обновляется. Все операции, кроме просмотра публичного профиля, требуют авторизации и проверки, что текущий пользователь имеет доступ к запрашиваемым данным (например, при просмотре чужих ставок или покупок возвращается ошибка). Модуль тесно взаимодействует с моделью лотов (LotsModel) для получения списков лотов пользователя и с базой данных для выполнения запросов к таблицам ставок и покупок.

Скриншоты страниц представлены в

Таким образом реализован функционал доступа к личному кабинету пользователя.

2.5. Модуль лотов и ставок

Модуль лотов и ставок является самым большим, так как представляет основную бизнес логику приложения. Модуль поделен на две части: публичную и защищенную. В публичную часть входит возможность просматривать лоты и всю информацию о них. В защищенной части производится все манипуляции с лотами и открывается возможность делать ставки.

Публичная часть модуля предоставляет всем посетителям возможность просматривать каталог лотов с фильтрацией по категориям, поиском по названию, городу или продавцу, а также сортировкой по цене, названию и времени окончания. Для удобства навигации реализована пагинация. При переходе на страницу конкретного лота отображается полная информация о товаре, включая фотографии, описание, текущую цену, лидера торгов, оставшееся время, способ оплаты и условия доставки. Здесь же выводится история всех сделанных ставок в хронологическом порядке.

Защищённая часть модуля доступна только авторизованным пользователям. Она включает создание новых лотов, редактирование и удаление собственных лотов. При создании лота необходимо указать название, город, начальную цену, количество, категорию, дату окончания торгов, описание, способ оплаты и условия доставки, а также загрузить минимум одно изображение. Все поля проходят валидацию на серверной стороне: проверяется положительность цены и количества, будущая дата окончания, существование категории, корректность выбранных способов оплаты, наличие изображений. При редактировании лота можно изменить любые параметры, кроме начальной цены, при этом старые изображения удаляются с диска, а новые сохраняются.

Участие в торгах реализовано через форму на странице лота. Пользователь вводит сумму, которая должна быть строго больше текущей цены, после чего система проверяет, не является ли участник продавцом, и не делал ли он уже ставку на этот лот. При успешной проверке создаётся запись о ставке, обновляется текущая цена и назначается новый лидер. Если пользователь передумал, он может отменить свою последнюю неперебитую ставку, и тогда

цена возвращается к предыдущему значению, а лидером становится предыдущий участник. Все изменения цен и списка ставок отображаются динамически благодаря использованию HTMX, который позволяет обновлять только нужные фрагменты страницы без полной перезагрузки.

После наступления времени окончания торгов лот автоматически переводится в закрытое состояние, и статус меняется на «ожидание подтверждения покупки». Победитель (лидер) видит на странице лота кнопку «Оставить заказ», которая позволяет зафиксировать покупку по текущей цене – при этом создаётся запись в таблице покупок, а статус лота становится «Выкуплен». Либо победитель может отказаться от покупки, тогда статус меняется на «отказ». В обоих случаях лот считается завершённым и больше не участвует в торгах.

Для повышения надёжности и предотвращения конкурентных гонок все операции со ставками и изменением данных лота выполняются в рамках одной транзакции, а при возникновении ошибок пользователю выводится понятное сообщение.

Дополнительно реализованы две фоновых службы: первая периодически проверяет истёкшие лоты и переводит их в закрытое состояние, чтобы продавец и покупатель могли завершить сделку, а вторая находит и удаляет с диска изображения-сироты, которые были прикреплены к удалённому лоту.

2.6. Модуль администратора

Модуль администратора предназначен для управления категориями лотов и доступен только пользователям с соответствующим флагом `is_admin`. На главной странице административной панели отображается список всех категорий с возможностью поиска по названию. Администратор может создавать новые категории, указывая уникальное имя, при этом проверяется отсутствие дубликатов. Для существующих категорий доступны редактирование (изменение названия) и удаление. При удалении категории связанные с ней лоты не удаляются, однако связь с категорией теряется, что может потребовать ручного переназначения. Все операции с категориями выполняются с использованием HTMX для динамического обновления списка без перезагрузки страницы. Ин-

терфейс администратора интуитивно понятен и позволяет быстро поддерживать актуальную структуру каталога, что упрощает навигацию для конечных пользователей.

2.7. Контейнеризация

Для обеспечения воспроизводимости окружения и упрощения развертывания реализована контейнеризация приложения с использованием Docker. Оркестрация контейнеров организована через Docker Compose, который объединяет два сервиса: сервер и базу данных.

Сборка выполняется одной командой `docker compose up --build`, что позволяет развернуть полную инфраструктуру приложения на любой системе с установленным Docker.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы разработана онлайн-платформа аукциона, реализованная в виде веб-приложения на платформе .NET с использованием ASP.NET Core MVC и Entity Framework Core, взаимодействующего с реляционной базой данных PostgreSQL. Клиентская часть построена на HTML, CSS и JavaScript с применением библиотеки HTMX, что позволило обеспечить динамическое обновление контента без полной перезагрузки страниц, сохраняя преимущества серверной отрисовки. Приложение развёртывается в контейнерах Docker, что гарантирует воспроизводимость среды и упрощает установку.

Ключевая функциональность охватывает регистрацию и аутентификацию пользователей на основе JWT-токенов с автоматическим обновлением сессии, создание и управление лотами (просмотр, фильтрация, поиск, сортировка, редактирование, удаление), участие в торгах с валидацией ставок (проверка превышения текущей цены, запрет ставок на свой лот, предотвращение дублирования), отмену последней неперебитой ставки, а также завершение торгов с выбором выкупа или отказа. Для пользователей реализован личный кабинет с просмотром созданных лотов, истории ставок и покупок; администраторы могут управлять категориями. Архитектурно проект демонстрирует применение принципов MVC, безопасное хранение данных (хеширование паролей bcrypt, JWT-аутентификация), строгую валидацию входящих данных и транзакционную обработку операций со ставками. Разработанное решение соответствует функциональным требованиям и готово к практическому использованию.

СПИСОК ИСТОЧНИКОВ

- 1) Иванова Г.С. Технология программирования: учебник для вузов. – М.: Изд-во МГТУ им. Н.Э. Баумана, 2003. – 320 с.
- 2) Рихтер Дж. CLR via C#. Программирование на платформе Microsoft .NET Framework 4.5 на языке C#. 4-е изд. – СПб.: Питер, 2013. – 896 с.
- 3) Албахари, Джозеф, Албахари, Бен. C# 9.0. Карманный справочник. : Пер. с англ. – СПб. : ООО “Диалектика”, 2021. – 256 с.
- 4) .NET 6 - ASP.NET core MVC - (Add database - SQLITE) [Электронный ресурс]. URL - <https://youtu.be/S7SdtcIr28s> (дата обращения: 09.07.2025)
- 5) Entity Properties [Электронный ресурс]. URL - <https://learn.microsoft.com/en-us/ef/core/modeling/entity-properties?tabs=data-annotations%2Cwith-nrt>
- 6) Secure Your .NET API in 15 Minutes: JWT Authentication Tutorial [Электронный ресурс]. URL - <https://youtu.be/6DWJIyirxzw> (дата обращения: 10.07.2025)
- 7) Best Practices for Secure Password Hashing in .NET (Stop Storing Passwords in Plain Text!) [Электронный ресурс]. URL - <https://www.youtube.com/watch?v=J4ix8Mhi3rs> (дата обращения: 10.07.2025)
- 8) Docker Compose with .NET 8, PostgreSQL, and Redis (step by step) [Электронный ресурс]. URL - <https://www.youtube.com/watch?v=WQFx2m5Ub9M&t=557s> (дата обращения: 11.07.2025)
- 9) Easy Email Verification in .NET: FluentEmail + Papercut [Электронный ресурс]. URL - <https://www.youtube.com/watch?v=KtCjH-1iCIk> (дата обращения: 11.07.2025)
- 10) Background tasks with hosted services in ASP.NET Core [Электронный ресурс]. URL - <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/host/hosted-services?view=aspnetcore-9.0&tabs=visual-studio> (дата обращения: 12.07.2025)

ПРИЛОЖЕНИЕ А. Диаграммы базы данных

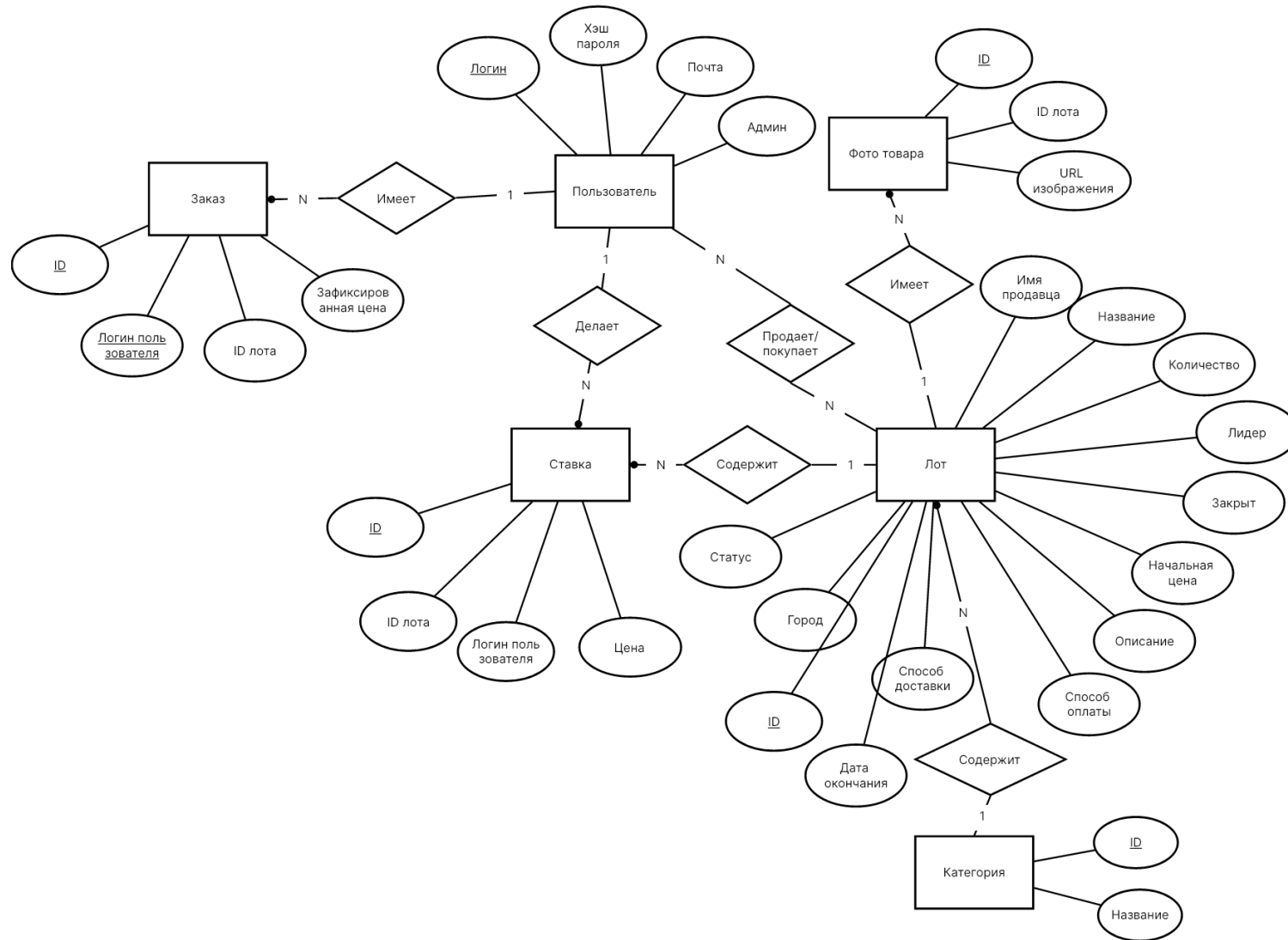


Рисунок А.1 – Диаграмма «сущность-связь»

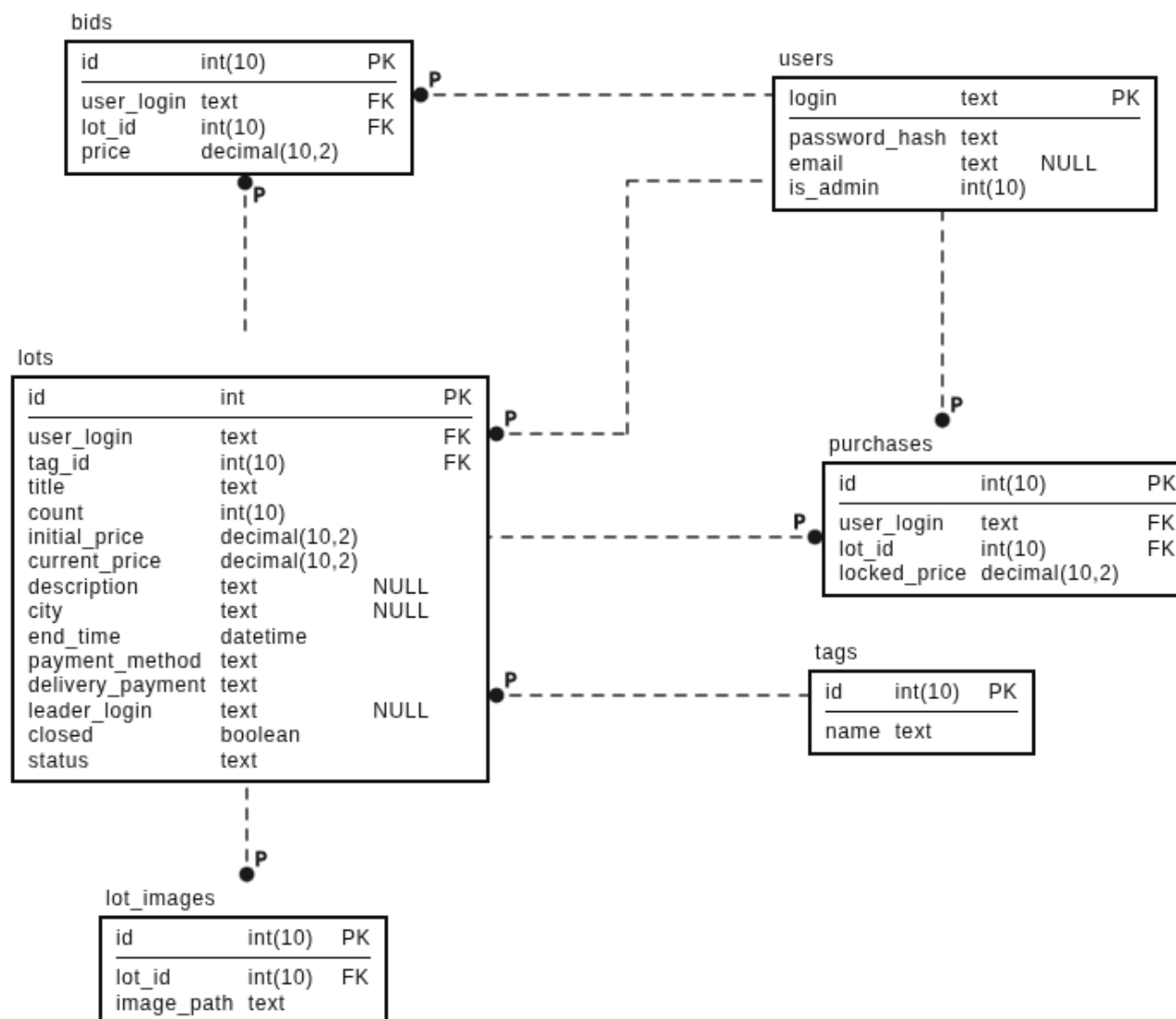


Рисунок А.2 – Полная атрибутивная модель

ПРИЛОЖЕНИЕ Б. Исходный код приложения

ПРИЛОЖЕНИЕ В. Исходный код приложения